

dff9-W4111-Spring-2026 - HW3.ipynb

Introduction

Homework Overview

There is one version of the homework that both the programming and non-programming tracks complete.

There are 4 categories of question:

1. Short answer questions testing **general knowledge** and the ability to reason about the course material.
2. **SQL** questions testing the ability to use SQL to produce data results from descriptions of the desired output.
3. Questions testing the ability to use **MongoDB** to produce results from descriptions of the desired output.
4. Questions testing the ability to use **Neo4j** to produce results from the descriptions of the desired output.

Submission Instructions

The homework is due at 11:59 PM on Sunday, 18-April-2026.

Please see [Ed discussion](#) for detailed submission instructions and for clarifications and answering questions.

Part 0 — Setup

iPython-SQL

```
In [1]: # Install the SQL magic extension and SQLAlchemy core  
%pip install ipython-sql sqlalchemy pandas pymysql
```

Requirement already satisfied: ipython-sql in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (0.5.0)

Requirement already satisfied: sqlalchemy in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (2.0.49)

Requirement already satisfied: pandas in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (2.3.3)

Requirement already satisfied: pymysql in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (1.1.2)

Requirement already satisfied: prettytable in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython-sql) (3.17.0)

Requirement already satisfied: ipython in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython-sql) (8.37.0)

Requirement already satisfied: sqlparse in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython-sql) (0.5.5)

Requirement already satisfied: six in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython-sql) (1.17.0)

Requirement already satisfied: ipython-genutils in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython-sql) (0.2.0)

Requirement already satisfied: typing-extensions>=4.6.0 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from sqlalchemy) (4.15.0)

Requirement already satisfied: numpy>=1.22.4 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from pandas) (2.2.6)

Requirement already satisfied: python-dateutil>=2.8.2 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from pandas) (2026.1.post1)

Requirement already satisfied: tzdata>=2022.7 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from pandas) (2026.1)

Requirement already satisfied: decorator in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython->ipython-sql) (5.2.1)

Requirement already satisfied: exceptiongroup in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython->ipython-sql) (1.3.1)

Requirement already satisfied: jedi>=0.16 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython->ipython-sql) (0.19.2)

Requirement already satisfied: matplotlib-inline in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython->ipython-sql) (0.2.1)

Requirement already satisfied: pexpect>4.3 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython->ipython-sql) (4.9.0)

Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython->ipython-sql) (3.0.52)

Requirement already satisfied: pygments>=2.4.0 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython->ipython-sql) (2.20.0)

Requirement already satisfied: stack_data in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython->ipython-sql) (0.6.3)

Requirement already satisfied: traitlets>=5.13.0 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from ipython->ipython-sql) (5.14.3)

Requirement already satisfied: wcwidth in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython->ipython-sql) (0.6.0)

Requirement already satisfied: parso<0.9.0,>=0.8.4 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from jedi>=0.16->ipython->ipython-sql) (0.8.6)

Requirement already satisfied: ptyprocess>=0.5 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from pexpect>4.3->ipython->ipython-sql) (0.7.0)

Requirement already satisfied: executing>=1.2.0 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from stack_data->ipython->ipython-sql) (2.2.1)

Requirement already satisfied: asttokens>=2.1.0 in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from stack_data->ipython->ipython-sql) (3.0.1)

Requirement already satisfied: pure_eval in /Users/minjae/miniconda3/envs/database_hw3/lib/python3.10/site-packages (from stack_data->ipython->ipython-sql) (0.2.3)

Note: you may need to restart the kernel to use updated packages.

In [2]: `%load_ext sql`

Set the proper root user ID and password for MySQL in the python cell below.

In [3]: `# mysql_root_user = 'root'
mysql_root_password = 'alswo4064'

mysql_url = f"mysql+pymysql://{mysql_root_user}:{mysql_root_password}@localhost"`

In [4]: `mysql_root_user = "root"
mysql_root_password = "alswo4064"
mysql_url = f"mysql+pymysql://{mysql_root_user}:{mysql_root_password}@localhost"`

In [5]: `mysql_url`

Out[5]: `'mysql+pymysql://root:alswo4064@localhost'`

In [6]: `%sql $mysql_url`

In [7]: `%config SqlMagic.style = '_DEPRECATED_DEFAULT'`

In [8]: `%sql select * from db_book.student where dept_name="Comp. Sci."`

`* mysql+pymysql://root:***@localhost
4 rows affected.`

Out[8]:

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
54321	Williams	Comp. Sci.	54
76543	Brown	Comp. Sci.	58

SQLAlchemy

```
In [9]: from sqlalchemy import create_engine, text
```

```
In [10]: engine = create_engine(mysql_url)
```

```
In [11]: # Raw SQL query
sql = text("SELECT * FROM db_book.student WHERE dept_name = :dept")

# Execute query
with engine.connect() as connection:
    result = connection.execute(sql, {"dept": "Comp. Sci."})

    for row in result:
        print(row)
```

```
('00128', 'Zhang', 'Comp. Sci.', Decimal('102'))
('12345', 'Shankar', 'Comp. Sci.', Decimal('32'))
('54321', 'Williams', 'Comp. Sci.', Decimal('54'))
('76543', 'Brown', 'Comp. Sci.', Decimal('58'))
```

Pandas

```
In [12]: import pandas
```

```
In [13]: result = pandas.read_sql(con=engine, sql="select * from db_book.student wher
```

```
In [14]: result
```

```
Out[14]:
```

	ID	name	dept_name	tot_cred
0	00128	Zhang	Comp. Sci.	102.0
1	12345	Shankar	Comp. Sci.	32.0
2	54321	Williams	Comp. Sci.	54.0
3	76543	Brown	Comp. Sci.	58.0

PyMySQL

```
In [15]: import pymysql
```

```
In [16]: conn = pymysql.connect(
    host="localhost",
    user=mysql_root_user,
    password=mysql_root_password,
    cursorclass=pymysql.cursors.DictCursor,
    autocommit=True
)
```

```
In [17]: cursor = conn.cursor()
```

```
In [18]: result = cursor.execute("select * from db_book.student where dept_name='Comp'
result = cursor.fetchall()
```

```
In [19]: result
```

```
Out[19]: [{'ID': '00128',
          'name': 'Zhang',
          'dept_name': 'Comp. Sci.',
          'tot_cred': Decimal('102')},
          {'ID': '12345',
          'name': 'Shankar',
          'dept_name': 'Comp. Sci.',
          'tot_cred': Decimal('32')},
          {'ID': '54321',
          'name': 'Williams',
          'dept_name': 'Comp. Sci.',
          'tot_cred': Decimal('54')},
          {'ID': '76543',
          'name': 'Brown',
          'dept_name': 'Comp. Sci.',
          'tot_cred': Decimal('58')}]
```

MongoDB

Test Connectivity

```
In [20]: import pymongo
```

```
In [21]: from pymongo import MongoClient, errors
```

```
In [22]: import json
```

```
In [23]: # #
          # # You will need to change this URL to connect to the MongoDB instance you
          # #
          # mongourl = "mongodb+srv://dbuser:sBJ1btaxRAWiqfm1@cluster0.t8qdk.mongodb.r
```

```
In [24]: from urllib.parse import quote_plus

          user = "mb4882"
          password = "Alsw0!3518?"
          host = "cluster0.xhp5mjw.mongodb.net"
          mongourl = f"mongodb+srv://{quote_plus(user)}:{quote_plus(password)}@{host}/
```

```
In [25]: client = MongoClient(mongourl)
```

```
In [26]: #
          # Your answer wrt to databases and collections will be different based on th
          # and collections you previously created. You connected successfully if you
          #
```

```

try:
    # Create a MongoClient with a short timeout for quick failure if unreachable
    # client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=5000)

    # Attempt to retrieve server information (forces a connection call)
    server_info = client.server_info()
    print("✅ Successfully connected to MongoDB Atlas!")
    print("Server Info:", server_info.get("version", "Unknown version"))

    # Optional: List available databases
    print("\nDatabases on this cluster:")
    for db_name in client.list_database_names():
        print(" -", db_name)

except errors.ServerSelectionTimeoutError as err:
    print("❌ Could not connect to MongoDB Atlas. Timeout occurred.")
    print("Error details:", err)
except errors.OperationFailure as err:
    print("❌ Authentication failed. Check your username/password.")
    print("Error details:", err)
except Exception as err:
    print("❌ An unexpected error occurred.")
    print("Error details:", err)

```

✅ Successfully connected to MongoDB Atlas!
 Server Info: 8.0.20

Databases on this cluster:

- classicmodels
- sample_mflix
- admin
- local

Load Classicmodels

In [27]: `import json`

In [28]: `with open('customers.json', 'r') as in_file:
 customers = json.load(in_file)`

In [29]: `customers[0:2]`

```
Out[29]: [{ 'customerNumber': 103,  
            'customerName': 'Atelier graphique',  
            'contact': { 'lastName': 'Schmitt', 'firstName': 'Carine ' },  
            'phone': '40.32.2555',  
            'address': { 'addressLine1': '54, rue Royale',  
                          'addressLine2': None,  
                          'city': 'Nantes',  
                          'state': None,  
                          'country': 'France',  
                          'postalCode': '44000' },  
            'salesRepNumber': 1370,  
            'creditLimit': 21000.0 },  
          { 'customerNumber': 112,  
            'customerName': 'Signal Gift Stores',  
            'contact': { 'lastName': 'King', 'firstName': 'Jean' },  
            'phone': '7025551838',  
            'address': { 'addressLine1': '8489 Strong St.',  
                          'addressLine2': None,  
                          'city': 'Las Vegas',  
                          'state': 'NV',  
                          'country': 'USA',  
                          'postalCode': '83030' },  
            'salesRepNumber': 1166,  
            'creditLimit': 71800.0 } ]
```

```
In [30]: len(customers)
```

```
Out[30]: 122
```

```
In [31]: db = client["classicmodels"]  
         db["orders"].drop()  
         db["customers"].drop()
```

```
In [32]: client['classicmodels']['customers'].insert_many(customers)
```


[illegible]

```
ObjectId('69e4119170342327b9695a2e'), ObjectId('69e4119170342327b9695a2f'),  
ObjectId('69e4119170342327b9695a30'), ObjectId('69e4119170342327b9695a31'),  
ObjectId('69e4119170342327b9695a32'), ObjectId('69e4119170342327b9695a33'),  
ObjectId('69e4119170342327b9695a34'), ObjectId('69e4119170342327b9695a35'),  
ObjectId('69e4119170342327b9695a36'), ObjectId('69e4119170342327b9695a37'),  
ObjectId('69e4119170342327b9695a38')], acknowledged=True)
```

```
In [33]: with open("orders.json", "r") as in_file:  
         orders = json.load(in_file)
```

```
In [34]: orders[0:2]
```

```
Out[34]: [{ 'orderNumber': 10100,
            'orderDate': '2003-01-06',
            'requiredDate': '2003-01-13',
            'shippedDate': '2003-01-10',
            'status': 'Shipped',
            'comments': None,
            'customerNumber': 363,
            'orderDetails': [{ 'productCode': 'S18_1749',
                                'quantityOrdered': 30,
                                'priceEach': 136.0,
                                'orderLineNumber': 3},
                              { 'productCode': 'S18_2248',
                                'quantityOrdered': 50,
                                'priceEach': 55.09,
                                'orderLineNumber': 2},
                              { 'productCode': 'S18_4409',
                                'quantityOrdered': 22,
                                'priceEach': 75.46,
                                'orderLineNumber': 4},
                              { 'productCode': 'S24_3969',
                                'quantityOrdered': 49,
                                'priceEach': 35.29,
                                'orderLineNumber': 1}]],
          { 'orderNumber': 10101,
            'orderDate': '2003-01-09',
            'requiredDate': '2003-01-18',
            'shippedDate': '2003-01-11',
            'status': 'Shipped',
            'comments': 'Check on availability.',
            'customerNumber': 128,
            'orderDetails': [{ 'productCode': 'S18_2325',
                                'quantityOrdered': 25,
                                'priceEach': 108.06,
                                'orderLineNumber': 4},
                              { 'productCode': 'S18_2795',
                                'quantityOrdered': 26,
                                'priceEach': 167.06,
                                'orderLineNumber': 1},
                              { 'productCode': 'S24_1937',
                                'quantityOrdered': 45,
                                'priceEach': 32.53,
                                'orderLineNumber': 3},
                              { 'productCode': 'S24_2022',
                                'quantityOrdered': 46,
                                'priceEach': 44.35,
                                'orderLineNumber': 2}]]}]
```

```
In [35]: len(orders)
```

```
Out[35]: 326
```

```
In [36]: client['classicmodels']['orders'].insert_many(orders)
```

[illegible]

[illegible]

Neo4j

```
In [37]: import neo4j
```

```
In [38]: # #
# # You must change to the information you downloaded when you created your
# #
# NE04J_URI="neo4j+ssc://c4fe1614.databases.neo4j.io"
# NE04J_USERNAME="c4fe1614"
# NE04J_PASSWORD="z2eolm_3MEkwYvicfs23Aotp3FacvYCKYw9wzkaBx0k"
# NE04J_DATABASE="c4fe1614"
# AURA_INSTANCEID="c4fe1614"
# AURA_INSTANCENAME="Instance01"
```

```
In [39]: # Wait 60 seconds before connecting using these details, or login to https://
NE04J_URI="neo4j+ssc://d640b480.databases.neo4j.io"
NE04J_USERNAME="d640b480"
NE04J_PASSWORD="G1fJc6zyHCKMwKGTvP5qSfs2jwcmF3V4w0Ad9hha_I0"
NE04J_DATABASE="d640b480"
AURA_INSTANCEID="d640b480"
AURA_INSTANCENAME="Instance01"
```

```
In [40]: from neo4j import GraphDatabase

# URI examples: "neo4j://localhost", "neo4j+s://xxx.databases.neo4j.io"

URL = NE04J_URI

AUTH = (NE04J_USERNAME, NE04J_PASSWORD)
"""Helper function"""
def run_query(q):

    with GraphDatabase.driver(NE04J_URI, auth=AUTH) as driver:
        driver.verify_connectivity()

        records, summary, keys = driver.execute_query(
            q,
            database_=NE04J_DATABASE,
        )

    # Loop through results and do something with them
    print("\nResult is")
    for record in records:
        print(record.data()) # obtain record as dict
```

```
In [41]: run_query("MATCH (p:Person)-[:FOLLOWS]->(:Person) RETURN p.name AS name")
```

```
Result is
{'name': 'Paul Blythe'}
{'name': 'Angela Scope'}
{'name': 'James Thompson'}
```

Part 1 — Written Questions

W1

Question

For relationships in the ER Model, briefly explain the concepts of **degree** and **cardinality**.

Answer

Degree refers to the number of entity sets that participate in a relationship. A binary relationship (degree 2) involves two entity sets and is the most common. A unary relation (degree 1) involves one entity set related to itself, and a ternary relationship (degree 3) involves three entity sets.

Cardinality refers to the maximum number of times an entity can participate in a relationship. The types are One-to-One (1:1), where one entity is associated with at most one other entity; One-to-Many (1:N), where one entity is associated with many others; and Many-to-Many (M:N), where many entities on both sides can be associated with many others.

W2

Question

Briefly explain the similarities and differences between **functions**, **procedures** and **triggers**.

Answer

Functions, procedures, and triggers are similar in that they are all routines stored in the database and written using a procedural language such as SQL/PSM.

However, they differ in how they are invoked and what they return. A function must return a value and is called explicitly within a SQL expression. A procedure does not need to return a value and is called explicitly using the CALL statement to perform a set of actions. A trigger, on the other hand, is never called explicitly – it is automatically executed in response to a database event such as INSERT, UPDATE, or DELETE.

W3

Question

Consider the table below and using your knowledge of the the **db_book sample DB** associated with the sample textbook, identify three functional dependencies in the table below.

	ID	name	dept_name	salary	building	budget
1	76766	Crick	Biology	72000.00	Watson	90000.00
2	10101	Srinivasan	Comp. Sci.	65000.00	Taylor	100000.00
3	45565	Katz	Comp. Sci.	75000.00	Taylor	100000.00
4	83821	Brandt	Comp. Sci.	92000.00	Taylor	100000.00
5	98345	Kim	Elec. Eng.	80000.00	Taylor	85000.00
6	12121	Wu	Finance	90000.00	Painter	120000.00
7	76543	Singh	Finance	80000.00	Painter	120000.00
8	32343	El Said	History	60000.00	Painter	50000.00
9	58583	Califieri	History	62000.00	Painter	50000.00
10	15151	Mozart	Music	40000.00	Packard	80000.00
11	22222	Einstein	Physics	95000.00	Watson	70000.00
12	33456	Gold	Physics	87000.00	Watson	70000.00

Answer

Looking at the table which combines instructor and department information, three functional dependencies are:

1. **ID -> name, dept_name, salary** – Each instructor ID uniquely determines the instructor's name, department, and salary.
2. **dept_name -> building, budget** – Each department name uniquely determines the building and budget. For example, all instructors in Comp. Sci share the same building (Taylor) and budget (100000).
3. **ID -> building, budget** – Since ID determines dept_name, and dept_name determines building and budget, ID also transitively determines building and budget.

W4

Question

Briefly compare **BCNF** and **3NF**.

Answer

Both BCNF and 3NF are normal forms that eliminate redundancy through decomposition. BCNF is stricter, requiring that for every functional dependency $X \rightarrow Y$, X must be a superkey, which eliminates all redundancy but may not preserve all functional dependencies. 3NF is slightly weaker, allowing exceptions when Y is part of a candidate key, which guarantees both lossless decomposition and dependency preservation but may leave some redundancy.

W5

Question

What are some benefits of RAID-5 compared to RAID-0 and RAID-1?

Answer

RAID-5 offers benefits over both RAID-0 and RAID-1. Compared to RAID-0, RAID-5 provides fault tolerance by using parity, so the system can recover data if a single disk fails, whereas RAID-0 has no redundancy and any disk failure results in total data loss. Compared to RAID-1, RAID-5 is more storage-efficient because it only requires one disk worth of space for parity across all disks, whereas RAID-1 mirrors all data and requires double the storage. Thus, RAID-5 balances both fault tolerance and storage efficiency.

W6

Question

For what types of query might columnar file organization be better than row oriented organization.

Answer

Columnar file organization is better than row-oriented organization for queries that access only a few attributes (columns) across a large number of records, such as aggregation queries (e.g. SUM, AVG, COUNT). Since only the relevant columns are read from disk rather than entire rows, I/O is significantly reduced. It is also beneficial for analytical and data warehousing workloads where read performance on specific attributes is more important than reading entire records.

W7

Question

For what types of query is LRU replacement a very bad buffer replacement policy?

Answer

LRU is a very bad buffer replacement policy for queries that perform sequential scans over large relations, such as a full table scan or a nested-loop join. In these cases, each page is accessed once and never reused immediately, so the most recently used page is actually the least likely to be needed again. LRU would evict useful pages while keeping pages that will not be referenced again, making it no better than a random replacement policy in this scenario.

S8

Question

How many **clustered indexes** can a table have? Why?

Answer

A table can have only one clustered index. This is because a clustered index determines the physical order in which records are stored on disk. Since the data can only be physically sorted in one way, only one clustered index is possible per table.

S9

Question

Briefly explain the concept of **index selectivity** and how it relates to **query processing/optimization**?

Answer

Index selectivity refers to how well an index can narrow down the number of records returned for a given query. A high selectivity index (e.g. a unique ID) means very few records match a search condition, making the index very effective at reducing I/O. A low selectivity index (e.g. a boolean column) means many records match, making the index less useful.

In **query processing and optimization**, the query optimizer uses selectivity to decide whether to use an index or perform a full table scan. If selectivity is high, using the index is efficient. If selectivity is low, a full table scan may be faster since the overhead of accessing the index may not be worth it.

S10

Question

Consider the following query for the db_book sample database associated with the recommended textbook.

```
select
    *
from
    student join takes using(ID)
where
    student.dept_name = 'Comp. Sci.';
```

What is an equivalent query that might be more efficient?

Answer

The original query joins all students with takes first, and then filters by department. A more efficient equivalent query applies the filter on student **before** the join, reducing the number of rows that need to be joined:

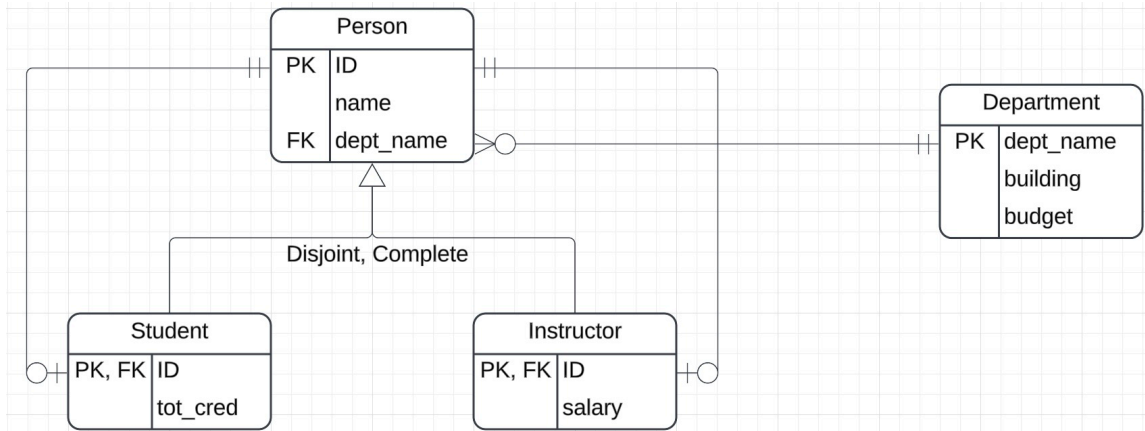
```
select *
from (select * from student where dept_name = 'Comp. Sci.')
as cs_students
join takes using(ID);
```

By filtering the student table first, only Comp. Sci. students are joined with takes, which reduces the size of the intermediate result and makes the join operation faster. This technique is called **selection pushdown** in query optimization.

Part 2 — SQL

S1

Consider the modification to the db_book schema depicted below. Write **DDL create table** and **create view** statements to implement the inheritance/specialization in SQL. Execute your statements in the cell below.

In [42]: `%%sql`

```

DROP SCHEMA IF EXISTS hw3;
CREATE SCHEMA hw3;
USE hw3;

-- =====
-- ① Parent table: Department
-- =====
CREATE TABLE Department (
    dept_name VARCHAR(20) NOT NULL,
    building VARCHAR(15),
    budget DECIMAL(12,2),
    PRIMARY KEY (dept_name)
);

-- =====
-- ② Superclass table: Person
-- =====
CREATE TABLE Person (
    ID VARCHAR(5),
    name VARCHAR(20) NOT NULL,
    dept_name VARCHAR(20),
    PRIMARY KEY (ID),
    FOREIGN KEY (dept_name) REFERENCES Department(dept_name)
);

-- =====
-- ③ Subclass: Student
-- =====
CREATE TABLE Student (
    ID VARCHAR(5),
    tot_cred DECIMAL(3,0),
    PRIMARY KEY (ID),
    FOREIGN KEY (ID) REFERENCES Person(ID)
    ON DELETE CASCADE
);

-- =====
-- ④ Subclass: Instructor
-- =====
CREATE TABLE Instructor (

```

```

        ID          VARCHAR(5),
        salary      DECIMAL(8,2),
        PRIMARY KEY (ID),
        FOREIGN KEY (ID) REFERENCES Person(ID)
            ON DELETE CASCADE
    );

-- =====
-- ⑤ Views: reconstruct full entity by joining
--    superclass and each subclass
-- =====

CREATE VIEW student_view AS
    SELECT p.ID, p.name, p.dept_name, s.tot_cred
    FROM   Person p
    JOIN   Student s ON p.ID = s.ID;

CREATE VIEW instructor_view AS
    SELECT p.ID, p.name, p.dept_name, i.salary
    FROM   Person p
    JOIN   Instructor i ON p.ID = i.ID;

```

```

* mysql+pymysql://root:***@localhost
6 rows affected.
1 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.

```

Out[42]: []

S2

Given your design above, write a **function** that computes a unique **ID** for a newly created **Person**. **Execute your function** and **show the results**.

In [43]: **%%sql**

```

-- Drop and recreate the function
DROP FUNCTION IF EXISTS new_person_id;

CREATE FUNCTION new_person_id()
RETURNS VARCHAR(5)
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE next_id INT;
    SELECT COALESCE(MAX(CAST(ID AS UNSIGNED)), 0) + 1
    INTO next_id
    FROM Person;

```

```
RETURN LPAD(next_id, 5, '0');
END;
```

```
* mysql+pymysql://root:***@localhost
0 rows affected.
0 rows affected.
```

Out [43]: []

```
In [44]: %sql
SELECT new_person_id() AS new_id;
```

```
* mysql+pymysql://root:***@localhost
1 rows affected.
```

Out [44]: new_id

00001

S3

Consider the following table computed from the tables `student`, `takes` and `course` in `db_book`

ID	name	tot_cred	computed_credits	courses	grades	course_credits
00128	Zhang	102		7 CS-101,CS-347	A,A-	4,3
12345	Shankar	32		14 CS-101,CS-190,CS-315,CS-347	C,A,A,A	4,4,3,3
19991	Brandt	80		3 HIS-351	B	3
23121	Chavez	110		3 FIN-201	C+	3
44553	Peltier	56		4 PHY-101	B-	4
45678	Levy	46		7 CS-101,CS-101,CS-319	F,B+,B	0,4,3
54321	Williams	54		8 CS-101,CS-190	A-,B+	4,4
55739	Sanchez	38		3 MU-199	A-	3
70557	Snow	0		0 <null>	<null>	<null>
76543	Brown	58		7 CS-101,CS-319	A,A	4,3
76653	Aoi	60		3 EE-181	C	3
98765	Bourikas	98		7 CS-101,CS-315	C-,B	4,3
98988	Tanaka	120		4 BIO-101,BIO-301	A,I	4,0

The columns are computed using the following rules:

- `ID`, `name` and `tot_cred` come from `student`.
- `courses` is a list of the `course_ids` for the courses a student took. The list is `null` if the student did not take any courses.
- `grades` is a list of grades the student earned in the courses and is computed from the following rules:
 - If the student did not take any course, the value is `null`.
 - If the student took a course but the grade was `null`, the value is `I`.
 - Otherwise the value is the student's grade in the course.
- `course_credits` is a list of the `course.credits` for the courses a student took.
 - The list is `null` if the student did not take any courses.
 - The student earned a `0` in the course if the grade is `F` or `I`.
 - Otherwise the list contains the the credits for the course.

Write and execute **SQL statement** that produces the table.

In [45]: `%%sql`

```

SELECT
    s.ID,
    s.name,
    s.tot_cred,

    -- computed_credits
    COALESCE(
        SUM(
            CASE
                WHEN t.grade IN ('F', 'I') OR t.grade IS NULL THEN 0
                ELSE c.credits
            END
        ), 0
    ) AS computed_credits,

    -- courses
    CASE
        WHEN COUNT(t.course_id) = 0 THEN NULL
        ELSE GROUP_CONCAT(
            t.course_id
            ORDER BY t.course_id, t.year, t.semester
            SEPARATOR ','
        )
    END AS courses,

    -- grades
    CASE
        WHEN COUNT(t.course_id) = 0 THEN NULL
        ELSE GROUP_CONCAT(
            CASE
                WHEN t.grade IS NULL THEN 'I'
                ELSE t.grade
            END
            ORDER BY t.course_id, t.year, t.semester
            SEPARATOR ','
        )
    END AS grades,

    -- course_credits
    CASE
        WHEN COUNT(t.course_id) = 0 THEN NULL
        ELSE GROUP_CONCAT(
            CASE
                WHEN t.grade IN ('F', 'I') OR t.grade IS NULL THEN 0
                ELSE c.credits
            END
            ORDER BY t.course_id, t.year, t.semester
            SEPARATOR ','
        )
    END AS course_credits

FROM db_book.student s
LEFT JOIN db_book.takes t ON s.ID = t.ID

```



```
LEFT JOIN db_book.course c ON t.course_id = c.course_id
```

```
GROUP BY s.ID, s.name, s.tot_cred
ORDER BY s.ID;
```

```
* mysql+pymysql://root:***@localhost
13 rows affected.
```

Out[45]:

ID	name	tot_cred	computed_credits	courses	grades	course_credits
00128	Zhang	102	7	CS-101,CS-347	A,A-	4,3
12345	Shankar	32	14	CS-101,CS-190,CS-315,CS-347	C,A,A,A	4,4,3,3
19991	Brandt	80	3	HIS-351	B	3
23121	Chavez	110	3	FIN-201	C+	3
44553	Peltier	56	4	PHY-101	B-	4
45678	Levy	46	7	CS-101,CS-101,CS-319	F,B+,B	0,4,3
54321	Williams	54	8	CS-101,CS-190	A-,B+	4,4
55739	Sanchez	38	3	MU-199	A-	3
70557	Snow	0	0	None	None	None
76543	Brown	58	7	CS-101,CS-319	A,A	4,3
76653	Aoi	60	3	EE-181	C	3
98765	Bourikas	98	7	CS-101,CS-315	C-,B	4,3
98988	Tanaka	120	4	BIO-101,BIO-301	A,I	4,0

Part 3 — MongoDB

Use the **database** and **collections** you created in the setup for these questions.

- **Cluster**
 - A MongoDB server unit created in Atlas.
- **database**
 - One of the several databases contained inside a cluster.
 - e.g.) classicmodels
 - How to search? (MongoDB Atlas -> Clusters -> Browse Collections)
- collection (≈ MySQL's table)
 - A group of data inside a database.

- e.g.) customers, orders
- document ($\approx$ MySQL's row(record))
 - One individual piece of data inside a collection.

M1

Produce a list of `customers` in "France." Your answer should only contain the fields `customerNumber`, `customerName`, `address`.

- filter
- projection

```
In [46]: """Connect to my MongoDB Atlas cluster."""
import pymongo
from pymongo import MongoClient, errors
import json

from urllib.parse import quote_plus

user = "mb4882"
password = "Alsw0!3518?"
host = "cluster0.xhp5mjw.mongodb.net"
mongourl = f"mongodb+srv://{quote_plus(user)}:{quote_plus(password)}@{host}/
```

```
In [47]: client = MongoClient(mongourl)
```

```
In [48]: # check whether it's connected to the MongoDB Atlas cluster
try:
    client.admin.command("ping")
    print("connected")
except Exception as e:
    print("not connected")
    print(e)
```

connected

```
In [49]: ## Pull out one document and check the actual field names
# client["classicmodels"]["customers"].find_one()
```

```
In [50]: ## Check what country values actually exist
# client["classicmodels"]["customers"].distinct("country")
```

```
In [51]: #
# Set your filter expression and projection expression below.
#
filter = {
    "address.country": "France"
}
projection = {
    "_id": 0,
    "customerNumber": 1,
```

```
"customerName": 1,  
"address": 1  
}
```

```
In [52]: #  
# Execute this cell to perform the query.  
#  
result = client["classicmodels"]["customers"].find(  
    filter = filter,  
    projection = projection  
)  
result = list(result)
```

```
In [53]: #  
# Execute this cell to validate your result  
#  
len(result)
```

Out[53]: 12

```
In [54]: #  
# Execute this cell to display and validate your results.  
#  
result[0:2]
```

```
Out[54]: [{'customerNumber': 103,  
  'customerName': 'Atelier graphique',  
  'address': {'addressLine1': '54, rue Royale',  
    'addressLine2': None,  
    'city': 'Nantes',  
    'state': None,  
    'country': 'France',  
    'postalCode': '44000'}},  
  {'customerNumber': 119,  
    'customerName': 'La Rochelle Gifts',  
    'address': {'addressLine1': '67, rue des Cinquante Otages',  
      'addressLine2': None,  
      'city': 'Nantes',  
      'state': None,  
      'country': 'France',  
      'postalCode': '44000'}}}]
```

M2

Write a (aggregation) pipeline that produces a **list of documents** of the form:

- orderNumber
- orderDate
- status
- requiredDate
- comments
- shippedDate

- `customerNumber`
- `productCode`
- `quantityOrdered`
- `priceEach`
- `orderLineNumber`

Your **list of documents** should be sorted by `orderNumber` and `orderLineNumber`.

```
In [ ]: # #
# # Enter your pipeline here.
# #
# pipeline = [
# ]
```

```
In [55]: client["classicmodels"]["orders"].find_one()
```

```
Out[55]: {'_id': ObjectId('69e4119270342327b9695a39'),
'orderNumber': 10100,
'orderDate': '2003-01-06',
'requiredDate': '2003-01-13',
'shippedDate': '2003-01-10',
'status': 'Shipped',
'comments': None,
'customerNumber': 363,
'orderDetails': [{'productCode': 'S18_1749',
'quantityOrdered': 30,
'priceEach': 136.0,
'orderLineNumber': 3},
{'productCode': 'S18_2248',
'quantityOrdered': 50,
'priceEach': 55.09,
'orderLineNumber': 2},
{'productCode': 'S18_4409',
'quantityOrdered': 22,
'priceEach': 75.46,
'orderLineNumber': 4},
{'productCode': 'S24_3969',
'quantityOrdered': 49,
'priceEach': 35.29,
'orderLineNumber': 1}]}
```

```
In [56]: pipeline = [
# Step 1: Unwind orderDetails array into rows
{
"$unwind": "$orderDetails"
},

# Step 2: Project only needed fields
{
"$project": {
"_id": 0,
"orderNumber": 1,
"orderDate": 1,
```

```

        "status": 1,
        "requiredDate": 1,
        "comments": 1,
        "shippedDate": 1,
        "customerNumber": 1,
        "productCode": "$orderDetails.productCode",
        "quantityOrdered": "$orderDetails.quantityOrdered",
        "priceEach": "$orderDetails.priceEach",
        "orderLineNumber": "$orderDetails.orderLineNumber"
    },
    },

    # Step 3: Sort by orderNumber, orderLineNumber
    {
        "$sort": {
            "orderNumber": 1,
            "orderLineNumber": 1
        }
    }
}
]

```

```

In [57]: #
# Execute this cell to run your query.
#
result = client['classicmodels']['orders'].aggregate(
    pipeline
)
result = list(result)

```

```

In [58]: #
# Execute this cell to validate your query.
#
len(result)

```

Out[58]: 2996

```

In [59]: #
# Execute this cell to validate and display your results.
#
result[0:20]

```

```
Out[59]: [{ 'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'productCode': 'S24_3969',
  'quantityOrdered': 49,
  'priceEach': 35.29,
  'orderLineNumber': 1},
{ 'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'productCode': 'S18_2248',
  'quantityOrdered': 50,
  'priceEach': 55.09,
  'orderLineNumber': 2},
{ 'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'productCode': 'S18_1749',
  'quantityOrdered': 30,
  'priceEach': 136.0,
  'orderLineNumber': 3},
{ 'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'productCode': 'S18_4409',
  'quantityOrdered': 22,
  'priceEach': 75.46,
  'orderLineNumber': 4},
{ 'orderNumber': 10101,
  'orderDate': '2003-01-09',
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-11',
  'status': 'Shipped',
  'comments': 'Check on availability.',
  'customerNumber': 128,
  'productCode': 'S18_2795',
  'quantityOrdered': 26,
  'priceEach': 167.06,
  'orderLineNumber': 1},
{ 'orderNumber': 10101,
```

```
'orderDate': '2003-01-09',
'requiredDate': '2003-01-18',
'shippedDate': '2003-01-11',
'status': 'Shipped',
'comments': 'Check on availability.',
'customerNumber': 128,
'productCode': 'S24_2022',
'quantityOrdered': 46,
'priceEach': 44.35,
'orderLineNumber': 2},
{'orderNumber': 10101,
'orderDate': '2003-01-09',
'requiredDate': '2003-01-18',
'shippedDate': '2003-01-11',
'status': 'Shipped',
'comments': 'Check on availability.',
'customerNumber': 128,
'productCode': 'S24_1937',
'quantityOrdered': 45,
'priceEach': 32.53,
'orderLineNumber': 3},
{'orderNumber': 10101,
'orderDate': '2003-01-09',
'requiredDate': '2003-01-18',
'shippedDate': '2003-01-11',
'status': 'Shipped',
'comments': 'Check on availability.',
'customerNumber': 128,
'productCode': 'S18_2325',
'quantityOrdered': 25,
'priceEach': 108.06,
'orderLineNumber': 4},
{'orderNumber': 10102,
'orderDate': '2003-01-10',
'requiredDate': '2003-01-18',
'shippedDate': '2003-01-14',
'status': 'Shipped',
'comments': None,
'customerNumber': 181,
'productCode': 'S18_1367',
'quantityOrdered': 41,
'priceEach': 43.13,
'orderLineNumber': 1},
{'orderNumber': 10102,
'orderDate': '2003-01-10',
'requiredDate': '2003-01-18',
'shippedDate': '2003-01-14',
'status': 'Shipped',
'comments': None,
'customerNumber': 181,
'productCode': 'S18_1342',
'quantityOrdered': 39,
'priceEach': 95.55,
'orderLineNumber': 2},
{'orderNumber': 10103,
'orderDate': '2003-01-29',
```

```
'requiredDate': '2003-02-07',
'shippedDate': '2003-02-02',
'status': 'Shipped',
'comments': None,
'customerNumber': 121,
'productCode': 'S24_2300',
'quantityOrdered': 36,
'priceEach': 107.34,
'orderLineNumber': 1},
{'orderNumber': 10103,
'orderId': '2003-01-29',
'requiredDate': '2003-02-07',
'shippedDate': '2003-02-02',
'status': 'Shipped',
'comments': None,
'customerNumber': 121,
'productCode': 'S18_2432',
'quantityOrdered': 22,
'priceEach': 58.34,
'orderLineNumber': 2},
{'orderNumber': 10103,
'orderId': '2003-01-29',
'requiredDate': '2003-02-07',
'shippedDate': '2003-02-02',
'status': 'Shipped',
'comments': None,
'customerNumber': 121,
'productCode': 'S32_1268',
'quantityOrdered': 31,
'priceEach': 92.46,
'orderLineNumber': 3},
{'orderNumber': 10103,
'orderId': '2003-01-29',
'requiredDate': '2003-02-07',
'shippedDate': '2003-02-02',
'status': 'Shipped',
'comments': None,
'customerNumber': 121,
'productCode': 'S10_4962',
'quantityOrdered': 42,
'priceEach': 119.67,
'orderLineNumber': 4},
{'orderNumber': 10103,
'orderId': '2003-01-29',
'requiredDate': '2003-02-07',
'shippedDate': '2003-02-02',
'status': 'Shipped',
'comments': None,
'customerNumber': 121,
'productCode': 'S18_4600',
'quantityOrdered': 36,
'priceEach': 98.07,
'orderLineNumber': 5},
{'orderNumber': 10103,
'orderId': '2003-01-29',
'requiredDate': '2003-02-07',
```



```

    'shippedDate': '2003-02-02',
    'status': 'Shipped',
    'comments': None,
    'customerNumber': 121,
    'productCode': 'S700_2824',
    'quantityOrdered': 42,
    'priceEach': 94.07,
    'orderLineNumber': 6},
  {'orderNumber': 10103,
    'orderDate': '2003-01-29',
    'requiredDate': '2003-02-07',
    'shippedDate': '2003-02-02',
    'status': 'Shipped',
    'comments': None,
    'customerNumber': 121,
    'productCode': 'S32_3522',
    'quantityOrdered': 45,
    'priceEach': 63.35,
    'orderLineNumber': 7},
  {'orderNumber': 10103,
    'orderDate': '2003-01-29',
    'requiredDate': '2003-02-07',
    'shippedDate': '2003-02-02',
    'status': 'Shipped',
    'comments': None,
    'customerNumber': 121,
    'productCode': 'S12_1666',
    'quantityOrdered': 27,
    'priceEach': 121.64,
    'orderLineNumber': 8},
  {'orderNumber': 10103,
    'orderDate': '2003-01-29',
    'requiredDate': '2003-02-07',
    'shippedDate': '2003-02-02',
    'status': 'Shipped',
    'comments': None,
    'customerNumber': 121,
    'productCode': 'S18_4668',
    'quantityOrdered': 41,
    'priceEach': 40.75,
    'orderLineNumber': 9},
  {'orderNumber': 10103,
    'orderDate': '2003-01-29',
    'requiredDate': '2003-02-07',
    'shippedDate': '2003-02-02',
    'status': 'Shipped',
    'comments': None,
    'customerNumber': 121,
    'productCode': 'S18_1097',
    'quantityOrdered': 35,
    'priceEach': 94.5,
    'orderLineNumber': 10}]

```

M3

The value of a line in `orderDetails` is `priceEach * quantityOrdered`.

The `totalValue` of an `order` is the sum over all line items of the value of the line.

Compute a list of documents of the form:

- `orderNumber`
- `totalValue`

Your answered should be sorted by `orderNumber`.

```
In [ ]: ##
        ## Enter your pipeline here.
        ##
        # pipeline = [
        # ]
```

```
In [76]: pipeline = [
        # Step 1: Unwind orderDetails array
        {
            "$unwind": "$orderDetails"
        },

        # Step 2: Group by order and compute totalValue
        {
            "$group": {
                "_id": "$orderNumber",
                "totalValue": {
                    "$sum": {
                        "$multiply": [
                            "$orderDetails.priceEach",
                            "$orderDetails.quantityOrdered"
                        ]
                    }
                }
            }
        },

        # Step 3: Rename _id to orderNumber
        {
            "$project": {
                "_id": 0,
                "orderNumber": "$_id",
                "totalValue": { "$round": ["$totalValue", 2] }
            }
        },

        # Step 4: Sort by orderNumber ascending
        {
            "$sort": {
                "orderNumber": 1
            }
        }
    ]
```

```
}  
]
```

```
In [77]: #  
# Execute this cell to run your query.  
#  
result = client['classicmodels']['orders'].aggregate(  
    pipeline  
)  
result = list(result)
```

```
In [78]: #  
# Execute this cell to validate your query.  
#  
len(result)
```

Out[78]: 326

```
In [79]: #  
# Execute this cell to validate and display your results.  
#  
result[0:20]
```

```
Out[79]: [{'orderNumber': 10100, 'totalValue': 10223.83},  
{'orderNumber': 10101, 'totalValue': 10549.01},  
{'orderNumber': 10102, 'totalValue': 5494.78},  
{'orderNumber': 10103, 'totalValue': 50218.95},  
{'orderNumber': 10104, 'totalValue': 40206.2},  
{'orderNumber': 10105, 'totalValue': 53959.21},  
{'orderNumber': 10106, 'totalValue': 52151.81},  
{'orderNumber': 10107, 'totalValue': 22292.62},  
{'orderNumber': 10108, 'totalValue': 51001.22},  
{'orderNumber': 10109, 'totalValue': 25833.14},  
{'orderNumber': 10110, 'totalValue': 48425.69},  
{'orderNumber': 10111, 'totalValue': 16537.85},  
{'orderNumber': 10112, 'totalValue': 7674.94},  
{'orderNumber': 10113, 'totalValue': 11044.3},  
{'orderNumber': 10114, 'totalValue': 33383.14},  
{'orderNumber': 10115, 'totalValue': 21665.98},  
{'orderNumber': 10116, 'totalValue': 1627.56},  
{'orderNumber': 10117, 'totalValue': 44380.15},  
{'orderNumber': 10118, 'totalValue': 3101.4},  
{'orderNumber': 10119, 'totalValue': 35826.33}]
```

Part 4 — Neo4j

Use Neo4j and the **sample Movie Data** for these queries.

- DB: Neo4j
- Data: sample Movie Data
- Query

```
In [80]: import neo4j

# Wait 60 seconds before connecting using these details, or login to https://
NEO4J_URI="neo4j+ssc://d640b480.databases.neo4j.io"
NEO4J_USERNAME="d640b480"
NEO4J_PASSWORD="G1fJc6zyHCKMwKGTvP5qSfs2jwcmF3V4w0Ad9hha_I0"
NEO4J_DATABASE="d640b480"
AURA_INSTANCEID="d640b480"
AURA_INSTANCENAME="Instance01"
```

```
In [81]: from neo4j import GraphDatabase

# URI examples: "neo4j://localhost", "neo4j+s://xxx.databases.neo4j.io"

URL = NEO4J_URI

AUTH = (NEO4J_USERNAME, NEO4J_PASSWORD)
"""Helper function"""
def run_query(q):

    with GraphDatabase.driver(NEO4J_URI, auth=AUTH) as driver:
        driver.verify_connectivity()

        records, summary, keys = driver.execute_query(
            q,
            database_=NEO4J_DATABASE,
        )

    # Loop through results and do something with them
    print("\nResult is")
    for record in records:
        print(record.data()) # obtain record as dict
```

```
In [82]: # Which nodes are in the DB
q = "CALL db.labels()"
run_query(q)
```

```
Result is
{'label': 'Movie'}
{'label': 'Person'}
```

```
In [83]: # What relations are in the DB
q = "CALL db.relationshipTypes()"
run_query(q)
```

```
Result is
{'relationshipType': 'ACTED_IN'}
{'relationshipType': 'DIRECTED'}
{'relationshipType': 'PRODUCED'}
{'relationshipType': 'WROTE'}
{'relationshipType': 'FOLLOWS'}
{'relationshipType': 'REVIEWED'}
```

```
In [84]: q = "MATCH (p:Person)-[:FOLLOWS]->(:Person) RETURN p.name AS name"
run_query(q)
```

Result is

```
{'name': 'Paul Blythe'}
{'name': 'Angela Scope'}
{'name': 'James Thompson'}
```

N1

Write a query that returns the **names of people** who directed movies and the **title of the movies** they directed.

```
In [85]: q = """
MATCH (p:Person)-[:DIRECTED]->(m:Movie)
RETURN p.name AS name, m.title AS title
"""
```

```
In [86]: run_query(q)
```

Result is

```
{'name': 'Lilly Wachowski', 'title': 'The Matrix'}
{'name': 'Lana Wachowski', 'title': 'The Matrix'}
{'name': 'Lilly Wachowski', 'title': 'The Matrix Reloaded'}
{'name': 'Lana Wachowski', 'title': 'The Matrix Reloaded'}
{'name': 'Lilly Wachowski', 'title': 'The Matrix Revolutions'}
{'name': 'Lana Wachowski', 'title': 'The Matrix Revolutions'}
{'name': 'Taylor Hackford', 'title': 'The Devil's Advocate'}
{'name': 'Rob Reiner', 'title': 'A Few Good Men'}
{'name': 'Tony Scott', 'title': 'Top Gun'}
{'name': 'Cameron Crowe', 'title': 'Jerry Maguire'}
{'name': 'Rob Reiner', 'title': 'Stand By Me'}
{'name': 'James L. Brooks', 'title': 'As Good as It Gets'}
{'name': 'Vincent Ward', 'title': 'What Dreams May Come'}
{'name': 'Scott Hicks', 'title': 'Snow Falling on Cedars'}
{'name': 'Nora Ephron', 'title': 'You've Got Mail'}
{'name': 'Nora Ephron', 'title': 'Sleepless in Seattle'}
{'name': 'John Patrick Stanley', 'title': 'Joe Versus the Volcano'}
{'name': 'Rob Reiner', 'title': 'When Harry Met Sally'}
{'name': 'Tom Hanks', 'title': 'That Thing You Do'}
{'name': 'Howard Deutch', 'title': 'The Replacements'}
{'name': 'Werner Herzog', 'title': 'RescueDawn'}
{'name': 'Mike Nichols', 'title': 'The Birdcage'}
{'name': 'Clint Eastwood', 'title': 'Unforgiven'}
{'name': 'Robert Longo', 'title': 'Johnny Mnemonic'}
{'name': 'Lilly Wachowski', 'title': 'Cloud Atlas'}
{'name': 'Lana Wachowski', 'title': 'Cloud Atlas'}
{'name': 'Tom Tykwer', 'title': 'Cloud Atlas'}
{'name': 'Ron Howard', 'title': 'The Da Vinci Code'}
{'name': 'James Marshall', 'title': 'V for Vendetta'}
{'name': 'Lilly Wachowski', 'title': 'Speed Racer'}
{'name': 'Lana Wachowski', 'title': 'Speed Racer'}
{'name': 'James Marshall', 'title': 'Ninja Assassin'}
{'name': 'Frank Darabont', 'title': 'The Green Mile'}
{'name': 'Ron Howard', 'title': 'Frost/Nixon'}
{'name': 'Danny DeVito', 'title': 'Hoffa'}
{'name': 'Ron Howard', 'title': 'Apollo 13'}
{'name': 'Jan de Bont', 'title': 'Twister'}
{'name': 'Robert Zemeckis', 'title': 'Cast Away'}
{'name': 'Milos Forman', 'title': 'One Flew Over the Cuckoo's Nest'}
{'name': 'Nancy Meyers', 'title': 'Something's Gotta Give'}
{'name': 'Chris Columbus', 'title': 'Bicentennial Man'}
{'name': 'Mike Nichols', 'title': 'Charlie Wilson's War'}
{'name': 'Robert Zemeckis', 'title': 'The Polar Express'}
{'name': 'Penny Marshall', 'title': 'A League of Their Own'}
```

N2

Write a query that computes the **director_name**, **movie_title**, and **actor_name** for each movie and order by director name.

```
In [87]: q = """
MATCH (d:Person)-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(a:Person)
RETURN d.name AS director_name, m.title AS movie_title, a.name AS actor_name
```

```
ORDER BY d.name  
.....
```

```
In [88]: run_query(q)
```

Result is

```
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Tom Cruise'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Cuba Gooding Jr.'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Renee Zellweger'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Kelly Preston'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Jerry O'Connell'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Jay Mohr'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Bonnie Hunt'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Regina King'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Jonathan Lipnicki'}
{'director_name': 'Chris Columbus', 'movie_title': 'Bicentennial Man', 'actor_name': 'Robin Williams'}
{'director_name': 'Chris Columbus', 'movie_title': 'Bicentennial Man', 'actor_name': 'Oliver Platt'}
{'director_name': 'Clint Eastwood', 'movie_title': 'Unforgiven', 'actor_name': 'Gene Hackman'}
{'director_name': 'Clint Eastwood', 'movie_title': 'Unforgiven', 'actor_name': 'Richard Harris'}
{'director_name': 'Clint Eastwood', 'movie_title': 'Unforgiven', 'actor_name': 'Clint Eastwood'}
{'director_name': 'Danny DeVito', 'movie_title': 'Hoffa', 'actor_name': 'Jack Nicholson'}
{'director_name': 'Danny DeVito', 'movie_title': 'Hoffa', 'actor_name': 'J.T. Walsh'}
{'director_name': 'Danny DeVito', 'movie_title': 'Hoffa', 'actor_name': 'Danny DeVito'}
{'director_name': 'Danny DeVito', 'movie_title': 'Hoffa', 'actor_name': 'John C. Reilly'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Bonnie Hunt'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'James Cromwell'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Tom Hanks'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Michael Clarke Duncan'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'David Morse'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Sam Rockwell'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Gary Sinise'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Patricia Clarkson'}
{'director_name': 'Howard Deutch', 'movie_title': 'The Replacements', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Howard Deutch', 'movie_title': 'The Replacements', 'actor_
```



```

name': 'Brooke Langton'}
{'director_name': 'Howard Deutch', 'movie_title': 'The Replacements', 'actor_
name': 'Gene Hackman'}
{'director_name': 'Howard Deutch', 'movie_title': 'The Replacements', 'actor_
name': 'Orlando Jones'}
{'director_name': 'James L. Brooks', 'movie_title': 'As Good as It Gets', 'ac
tor_name': 'Jack Nicholson'}
{'director_name': 'James L. Brooks', 'movie_title': 'As Good as It Gets', 'ac
tor_name': 'Cuba Gooding Jr.'}
{'director_name': 'James L. Brooks', 'movie_title': 'As Good as It Gets', 'ac
tor_name': 'Helen Hunt'}
{'director_name': 'James L. Brooks', 'movie_title': 'As Good as It Gets', 'ac
tor_name': 'Greg Kinnear'}
{'director_name': 'James Marshall', 'movie_title': 'V for Vendetta', 'actor_n
ame': 'Hugo Weaving'}
{'director_name': 'James Marshall', 'movie_title': 'V for Vendetta', 'actor_n
ame': 'Natalie Portman'}
{'director_name': 'James Marshall', 'movie_title': 'V for Vendetta', 'actor_n
ame': 'Stephen Rea'}
{'director_name': 'James Marshall', 'movie_title': 'V for Vendetta', 'actor_n
ame': 'John Hurt'}
{'director_name': 'James Marshall', 'movie_title': 'V for Vendetta', 'actor_n
ame': 'Ben Miles'}
{'director_name': 'James Marshall', 'movie_title': 'Ninja Assassin', 'actor_n
ame': 'Rick Yune'}
{'director_name': 'James Marshall', 'movie_title': 'Ninja Assassin', 'actor_n
ame': 'Ben Miles'}
{'director_name': 'James Marshall', 'movie_title': 'Ninja Assassin', 'actor_n
ame': 'Rain'}
{'director_name': 'James Marshall', 'movie_title': 'Ninja Assassin', 'actor_n
ame': 'Naomie Harris'}
{'director_name': 'Jan de Bont', 'movie_title': 'Twister', 'actor_name': 'Hel
en Hunt'}
{'director_name': 'Jan de Bont', 'movie_title': 'Twister', 'actor_name': 'Zac
h Grenier'}
{'director_name': 'Jan de Bont', 'movie_title': 'Twister', 'actor_name': 'Bil
l Paxton'}
{'director_name': 'Jan de Bont', 'movie_title': 'Twister', 'actor_name': 'Phi
lip Seymour Hoffman'}
{'director_name': 'John Patrick Stanley', 'movie_title': 'Joe Versus the Volc
ano', 'actor_name': 'Meg Ryan'}
{'director_name': 'John Patrick Stanley', 'movie_title': 'Joe Versus the Volc
ano', 'actor_name': 'Tom Hanks'}
{'director_name': 'John Patrick Stanley', 'movie_title': 'Joe Versus the Volc
ano', 'actor_name': 'Nathan Lane'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix', 'actor_nam
e': 'Keanu Reeves'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix', 'actor_nam
e': 'Carrie-Anne Moss'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix', 'actor_nam
e': 'Laurence Fishburne'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix', 'actor_nam
e': 'Hugo Weaving'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix', 'actor_nam
e': 'Emil Eifrem'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Reloaded', 'ac

```

```

tor_name': 'Keanu Reeves'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Carrie-Anne Moss'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Laurence Fishburne'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Carrie-Anne Moss'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Laurence Fishburne'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Tom Hanks'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Halle Berry'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Jim Broadbent'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Ben Miles'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Emile Hirsch'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'John Goodman'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Susan Sarandon'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Matthew Fox'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Christina Ricci'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Rain'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Carrie-Anne Moss'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Laurence Fishburne'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Emil Eifrem'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Carrie-Anne Moss'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Laurence Fishburne'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Revolutions',

```

```

'actor_name': 'Keanu Reeves'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Revolutions',
'actor_name': 'Carrie-Anne Moss'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Revolutions',
'actor_name': 'Laurence Fishburne'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Revolutions',
'actor_name': 'Hugo Weaving'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Tom Hanks'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Halle Berry'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Jim Broadbent'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Ben Miles'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Emile Hirsch'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'John Goodman'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Susan Sarandon'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Matthew Fox'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Christina Ricci'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Rain'}
{'director_name': 'Mike Nichols', 'movie_title': 'The Birdcage', 'actor_name': 'Robin Williams'}
{'director_name': 'Mike Nichols', 'movie_title': 'The Birdcage', 'actor_name': 'Nathan Lane'}
{'director_name': 'Mike Nichols', 'movie_title': 'The Birdcage', 'actor_name': 'Gene Hackman'}
{'director_name': 'Mike Nichols', 'movie_title': 'Charlie Wilson's War', 'actor_name': 'Tom Hanks'}
{'director_name': 'Mike Nichols', 'movie_title': 'Charlie Wilson's War', 'actor_name': 'Philip Seymour Hoffman'}
{'director_name': 'Mike Nichols', 'movie_title': 'Charlie Wilson's War', 'actor_name': 'Julia Roberts'}
{'director_name': 'Milos Forman', 'movie_title': 'One Flew Over the Cuckoo's Nest', 'actor_name': 'Jack Nicholson'}
{'director_name': 'Milos Forman', 'movie_title': 'One Flew Over the Cuckoo's Nest', 'actor_name': 'Danny DeVito'}
{'director_name': 'Nancy Meyers', 'movie_title': 'Something's Gotta Give', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Nancy Meyers', 'movie_title': 'Something's Gotta Give', 'actor_name': 'Jack Nicholson'}
{'director_name': 'Nancy Meyers', 'movie_title': 'Something's Gotta Give', 'actor_name': 'Diane Keaton'}
{'director_name': 'Nora Ephron', 'movie_title': 'You've Got Mail', 'actor_name': 'Meg Ryan'}
{'director_name': 'Nora Ephron', 'movie_title': 'You've Got Mail', 'actor_name': 'Greg Kinnear'}
{'director_name': 'Nora Ephron', 'movie_title': 'You've Got Mail', 'actor_name': 'Greg Kinnear'}

```

```

e': 'Tom Hanks'}
{'director_name': 'Nora Ephron', 'movie_title': "You've Got Mail", 'actor_name': 'Parker Posey'}
{'director_name': 'Nora Ephron', 'movie_title': "You've Got Mail", 'actor_name': 'Dave Chappelle'}
{'director_name': 'Nora Ephron', 'movie_title': "You've Got Mail", 'actor_name': 'Steve Zahn'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': 'Meg Ryan'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': 'Tom Hanks'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': 'Rita Wilson'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': 'Bill Pullman'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': 'Victor Garber'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': "Rosie O'Donnell"}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': 'Tom Hanks'}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': "Rosie O'Donnell"}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': 'Bill Paxton'}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': 'Madonna'}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': 'Geena Davis'}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': 'Lori Petty'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Tom Cruise'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Jack Nicholson'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Demi Moore'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Kevin Bacon'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Kiefer Sutherland'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Noah Wyle'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Cuba Gooding Jr.'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Kevin Pollak'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'J.T. Walsh'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'James Marshall'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Christopher Guest'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Aaron Sorkin'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name':

```

```

'Kiefer Sutherland'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name':
'Jerry O'Connell'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name':
'River Phoenix'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name':
'Corey Feldman'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name':
'Wil Wheaton'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name':
'John Cusack'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name':
'Marshall Bell'}
{'director_name': 'Rob Reiner', 'movie_title': 'When Harry Met Sally', 'actor
_name': 'Meg Ryan'}
{'director_name': 'Rob Reiner', 'movie_title': 'When Harry Met Sally', 'actor
_name': 'Billy Crystal'}
{'director_name': 'Rob Reiner', 'movie_title': 'When Harry Met Sally', 'actor
_name': 'Carrie Fisher'}
{'director_name': 'Rob Reiner', 'movie_title': 'When Harry Met Sally', 'actor
_name': 'Bruno Kirby'}
{'director_name': 'Robert Longo', 'movie_title': 'Johnny Mnemonic', 'actor_na
me': 'Keanu Reeves'}
{'director_name': 'Robert Longo', 'movie_title': 'Johnny Mnemonic', 'actor_na
me': 'Takeshi Kitano'}
{'director_name': 'Robert Longo', 'movie_title': 'Johnny Mnemonic', 'actor_na
me': 'Dina Meyer'}
{'director_name': 'Robert Longo', 'movie_title': 'Johnny Mnemonic', 'actor_na
me': 'Ice-T'}
{'director_name': 'Robert Zemeckis', 'movie_title': 'Cast Away', 'actor_nam
e': 'Helen Hunt'}
{'director_name': 'Robert Zemeckis', 'movie_title': 'Cast Away', 'actor_nam
e': 'Tom Hanks'}
{'director_name': 'Robert Zemeckis', 'movie_title': 'The Polar Express', 'act
or_name': 'Tom Hanks'}
{'director_name': 'Ron Howard', 'movie_title': 'The Da Vinci Code', 'actor_na
me': 'Tom Hanks'}
{'director_name': 'Ron Howard', 'movie_title': 'The Da Vinci Code', 'actor_na
me': 'Ian McKellen'}
{'director_name': 'Ron Howard', 'movie_title': 'The Da Vinci Code', 'actor_na
me': 'Audrey Tautou'}
{'director_name': 'Ron Howard', 'movie_title': 'The Da Vinci Code', 'actor_na
me': 'Paul Bettany'}
{'director_name': 'Ron Howard', 'movie_title': 'Frost/Nixon', 'actor_name':
'Kevin Bacon'}
{'director_name': 'Ron Howard', 'movie_title': 'Frost/Nixon', 'actor_name':
'Sam Rockwell'}
{'director_name': 'Ron Howard', 'movie_title': 'Frost/Nixon', 'actor_name':
'Frank Langella'}
{'director_name': 'Ron Howard', 'movie_title': 'Frost/Nixon', 'actor_name':
'Michael Sheen'}
{'director_name': 'Ron Howard', 'movie_title': 'Frost/Nixon', 'actor_name':
'Oliver Platt'}
{'director_name': 'Ron Howard', 'movie_title': 'Apollo 13', 'actor_name': 'Ke
vin Bacon'}
{'director_name': 'Ron Howard', 'movie_title': 'Apollo 13', 'actor_name': 'To

```

```

m Hanks'}}
{'director_name': 'Ron Howard', 'movie_title': 'Apollo 13', 'actor_name': 'Ga
ry Sinise'}}
{'director_name': 'Ron Howard', 'movie_title': 'Apollo 13', 'actor_name': 'Ed
Harris'}}
{'director_name': 'Ron Howard', 'movie_title': 'Apollo 13', 'actor_name': 'Bi
ll Paxton'}}
{'director_name': 'Scott Hicks', 'movie_title': 'Snow Falling on Cedars', 'ac
tor_name': 'Max von Sydow'}}
{'director_name': 'Scott Hicks', 'movie_title': 'Snow Falling on Cedars', 'ac
tor_name': 'Ethan Hawke'}}
{'director_name': 'Scott Hicks', 'movie_title': 'Snow Falling on Cedars', 'ac
tor_name': 'Rick Yune'}}
{'director_name': 'Scott Hicks', 'movie_title': 'Snow Falling on Cedars', 'ac
tor_name': 'James Cromwell'}}
{'director_name': 'Taylor Hackford', 'movie_title': 'The Devil's Advocate',
'actor_name': 'Keanu Reeves'}}
{'director_name': 'Taylor Hackford', 'movie_title': 'The Devil's Advocate',
'actor_name': 'Charlize Theron'}}
{'director_name': 'Taylor Hackford', 'movie_title': 'The Devil's Advocate',
'actor_name': 'Al Pacino'}}
{'director_name': 'Tom Hanks', 'movie_title': 'That Thing You Do', 'actor_nam
e': 'Charlize Theron'}}
{'director_name': 'Tom Hanks', 'movie_title': 'That Thing You Do', 'actor_nam
e': 'Tom Hanks'}}
{'director_name': 'Tom Hanks', 'movie_title': 'That Thing You Do', 'actor_nam
e': 'Liv Tyler'}}
{'director_name': 'Tom Tykwer', 'movie_title': 'Cloud Atlas', 'actor_name':
'Hugo Weaving'}}
{'director_name': 'Tom Tykwer', 'movie_title': 'Cloud Atlas', 'actor_name':
'Tom Hanks'}}
{'director_name': 'Tom Tykwer', 'movie_title': 'Cloud Atlas', 'actor_name':
'Halle Berry'}}
{'director_name': 'Tom Tykwer', 'movie_title': 'Cloud Atlas', 'actor_name':
'Jim Broadbent'}}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Tom
Cruise'}}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Kell
y McGillis'}}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Val
Kilmer'}}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Anth
ony Edwards'}}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Tom
Skerritt'}}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Meg
Ryan'}}
{'director_name': 'Vincent Ward', 'movie_title': 'What Dreams May Come', 'act
or_name': 'Cuba Gooding Jr.'}}
{'director_name': 'Vincent Ward', 'movie_title': 'What Dreams May Come', 'act
or_name': 'Annabella Sciorra'}}
{'director_name': 'Vincent Ward', 'movie_title': 'What Dreams May Come', 'act
or_name': 'Max von Sydow'}}
{'director_name': 'Vincent Ward', 'movie_title': 'What Dreams May Come', 'act
or_name': 'Werner Herzog'}}
{'director_name': 'Vincent Ward', 'movie_title': 'What Dreams May Come', 'act

```

```

or_name': 'Robin Williams'}
{'director_name': 'Werner Herzog', 'movie_title': 'RescueDawn', 'actor_name':
'Marshall Bell'}
{'director_name': 'Werner Herzog', 'movie_title': 'RescueDawn', 'actor_name':
'Steve Zahn'}
{'director_name': 'Werner Herzog', 'movie_title': 'RescueDawn', 'actor_name':
'Christian Bale'}
{'director_name': 'Werner Herzog', 'movie_title': 'RescueDawn', 'actor_name':
'Zach Grenier'}

```

N3

Return a list of **all people who acted in a movie with Tim Hanks**. Order the list by movie.title.

```

In [ ]: # q = f"""
# MATCH (tim:Person {name: "Tom Hanks"})-[:ACTED_IN]->(m:Movie)<-[:ACTED_I
# WHERE p.name <> "Tom Hanks"
# RETURN p.name AS name, m.title AS movie_title
# ORDER BY m.title
# """

```

```

In [89]: q = """
MATCH (tim:Person {name: "Tom Hanks"})-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-
WHERE p.name <> "Tom Hanks"
RETURN p.name AS name, m.title AS movie_title
ORDER BY m.title
"""

```

```

In [90]: run_query(q)

```

Result is

```
{'name': 'Rosie O'Donnell', 'movie_title': 'A League of Their Own'}
{'name': 'Bill Paxton', 'movie_title': 'A League of Their Own'}
{'name': 'Madonna', 'movie_title': 'A League of Their Own'}
{'name': 'Geena Davis', 'movie_title': 'A League of Their Own'}
{'name': 'Lori Petty', 'movie_title': 'A League of Their Own'}
{'name': 'Kevin Bacon', 'movie_title': 'Apollo 13'}
{'name': 'Gary Sinise', 'movie_title': 'Apollo 13'}
{'name': 'Ed Harris', 'movie_title': 'Apollo 13'}
{'name': 'Bill Paxton', 'movie_title': 'Apollo 13'}
{'name': 'Helen Hunt', 'movie_title': 'Cast Away'}
{'name': 'Philip Seymour Hoffman', 'movie_title': 'Charlie Wilson's War'}
{'name': 'Julia Roberts', 'movie_title': 'Charlie Wilson's War'}
{'name': 'Hugo Weaving', 'movie_title': 'Cloud Atlas'}
{'name': 'Halle Berry', 'movie_title': 'Cloud Atlas'}
{'name': 'Jim Broadbent', 'movie_title': 'Cloud Atlas'}
{'name': 'Meg Ryan', 'movie_title': 'Joe Versus the Volcano'}
{'name': 'Nathan Lane', 'movie_title': 'Joe Versus the Volcano'}
{'name': 'Meg Ryan', 'movie_title': 'Sleepless in Seattle'}
{'name': 'Rita Wilson', 'movie_title': 'Sleepless in Seattle'}
{'name': 'Bill Pullman', 'movie_title': 'Sleepless in Seattle'}
{'name': 'Victor Garber', 'movie_title': 'Sleepless in Seattle'}
{'name': 'Rosie O'Donnell', 'movie_title': 'Sleepless in Seattle'}
{'name': 'Charlize Theron', 'movie_title': 'That Thing You Do'}
{'name': 'Liv Tyler', 'movie_title': 'That Thing You Do'}
{'name': 'Ian McKellen', 'movie_title': 'The Da Vinci Code'}
{'name': 'Audrey Tautou', 'movie_title': 'The Da Vinci Code'}
{'name': 'Paul Bettany', 'movie_title': 'The Da Vinci Code'}
{'name': 'Bonnie Hunt', 'movie_title': 'The Green Mile'}
{'name': 'James Cromwell', 'movie_title': 'The Green Mile'}
{'name': 'Michael Clarke Duncan', 'movie_title': 'The Green Mile'}
{'name': 'David Morse', 'movie_title': 'The Green Mile'}
{'name': 'Sam Rockwell', 'movie_title': 'The Green Mile'}
{'name': 'Gary Sinise', 'movie_title': 'The Green Mile'}
{'name': 'Patricia Clarkson', 'movie_title': 'The Green Mile'}
{'name': 'Meg Ryan', 'movie_title': 'You've Got Mail'}
{'name': 'Greg Kinnear', 'movie_title': 'You've Got Mail'}
{'name': 'Parker Posey', 'movie_title': 'You've Got Mail'}
{'name': 'Dave Chappelle', 'movie_title': 'You've Got Mail'}
{'name': 'Steve Zahn', 'movie_title': 'You've Got Mail'}
```

In []: